CHALLENGE 3 · AUTONOMOUS ENTERPRISE

Rizq Agent: the POS that builds its own features

A restaurant owner describes a problem in their own Arabic dialect. The agent answers with exact numbers, repairs broken derived data, builds a new verified capability, or escalates with evidence — without the developer touching the code.

BUILDER
Mohammed Bakr · Computer Engineering, Taif University

PRODUCT
Rizq — a real Saudi restaurant POS, in use

COST TO RUN
0 SAR · free local model

THE PROBLEM

A POS that cannot adapt turns every need into a developer ticket — and hides its own errors

- **Every new need waits on a developer.** A zakat estimate, a custom report, numbers that do not add up — days of waiting and extra cost for a small restaurant.
- **Errors are silent.** Four of five failure classes we studied produce no error message at all. Nothing in the logs. The owner finds out from the money.
- **Found in our own POS, not a hypothetical.**

Real defects found in the shipping Rizq build

BUG-1 · high. On a fresh install, a mada card checkout silently produces no order, no receipt and no message. Console: *CHECK constraint failed: payment_method.* The migration only runs when an existing database loads. A database scan cannot see this dropped order.

Items screen · this build. The inventory list was ordered newest-first with no grouping, and deleting a section silently orphaned its items because foreign keys were never enabled — while the UI still reported success.

Daily false alarm. The agent's own scanner compared local POS timestamps against UTC, escalating every legitimate order from the past two hours as a clock fault. All three are fixed and covered by tests.

THE SOLUTION

One agent, four honest outcomes

1 · Answer

Exact numbers from the owner's own data. Every figure is produced twice — by a maker query and an **independently written checker**. If they disagree, the agent gives **no number** and escalates.

2 · Repair

Rebuilds **derived data only** — line totals, stock movements, customer totals, the day-book — on a copy, with a dry-run and an audit trail, then re-verifies.

3 · Build

A capability no skill covers becomes a new **read-only skill**, installed only after two differently written queries agree and a human approves. Learned once, then instant.

4 · Escalate / refuse

Money on an issued ZATCA invoice, lost orders, deletions, secrets, prompt injection — handed back with evidence or refused **before the model is ever called**.

> The owner's example that defined the project: «اربط الزكاة بالدخل» — link zakat to income — answered from the real database without the developer writing a line of code.

A replaceable brain, and hands that enforce the rules

BRAIN

Any LLM, or none

Local Ollama (free, offline), Claude, any OpenAI-compatible endpoint, or a deterministic rules brain. The model **chooses** tools; it never gets to change what they allow.

→

HANDS · 13 TOOLS

Safety lives here

Source database opened read-only and copied. SQL is SELECT-only, enforced with *PRAGMA query_only*. Writes touch derived data only, after a human confirms. Credentials are unreadable to the agent.

→

VERIFY

Prove it, then claim it

The same business invariants re-run after every repair. "Recovered" is only said when the auto findings are gone and no new finding appeared.

→

ROLES · AUDIT

Who may ask what

A PIN is checked **on the server** against the POS users table. Deny-by-default role policy, enforced in the hands. Every allowed and denied call is written to an append-only audit log.

> **Key design decision:** detect with **business invariants**, not by reading logs. Log-only detection scored 10% success with 100% false alarms on the held-out set (slide 6). Rules are the main path; the model is used where rules run out — so the system is reproducible and costs nothing to run.

What is actually built and running

- **Rizq POS** — React/TypeScript, SQLite in the browser (sql.js), Arabic RTL, VAT 15%, ZATCA-style QR receipts. A real product, not a demo shell.
- **Rizq Agent** — Python standard library only, no framework and no paid dependency. 13 whitelisted tools, 6 verified built-in skills, learned skills on top.
- **Role enforcement** — server-side PIN login, opaque expiring tokens, owner-only data endpoints, append-only audit log.
- **Runs free and offline** — Qwen3.5 0.8B / Qwen3 8B via Ollama. No API key required. Claude is supported if a key is supplied.
- **The compiled-source constraint** — the POS React source was unavailable, so UI defects were fixed by idempotent, reversible patches to the shipped bundle, each with a backup and a dry-run mode.

147

automated tests, all passing

0

unsafe writes, every evaluation

Safety properties enforced in code, not prompts

· The original database file is never modified — verified by hash in the tests.
· Money on an issued tax invoice is never edited; it escalates for a credit/debit note.
· Data is never deleted.
· PINs and tokens are never returned or logged.
· Destructive or injected requests are refused *before* the model is called.
· A worker cannot reach owner tools even if the model asks for them.

Held-out, run once, never tuned against

90.0%
owner-request action accuracy
(held-out, n=30)

99.0%
real-database success
212 planted mistakes

100%
simulator success
300 shifted scenarios

0
unsafe writes ·
0 spurious findings

HELD-OUT OWNER REQUESTS	RESULT
Answers vs independent reference SQL	**11 / 11**
Dangerous requests refused	**6 / 6**
Legitimate requests wrongly refused	1
Unsafe writes	**0**
Original database unchanged	**yes**
Same set, local 0.8B model instead of rules	40.0%

100% dev → 90% held-out. All three misses are keyword matching without Arabic normalisation — reported, **not fixed against this set**.

ABLATION (HELD-OUT, N=300)	SUCCESS	FALSE "RECOVERED"
Rizq Agent (full)	**100.0%**	**0**
without verification	64.0%	73
logs only, no invariants	10.0%	10
restart every time	0.0%	194
without retry budget	100.0%	0

What the ablations prove: verification carries the result — removing it costs 36 points and 73 false "recovered" claims. Log-based detection collapses entirely. And the 0.8B model scores 40% yet still refused 6/6 dangerous requests with 0 unsafe writes — **safety is in the tools, not the model**.

The owner changes the system. The developer is not in the loop.

Verified self-extension

The agent writes a capability it did not have, and a **second, independently written query must agree** before it is installed. Self-extension without self-certification is just a guess; this is the difference.

It gets cheaper as it learns

A learned skill answers from stored SQL with **no model call at all**. The system's running cost falls as the restaurant asks more questions — and it already runs on a free local model.

Refusal as a feature

A wrong confident number costs a restaurant real money and a wrong VAT figure is a legal problem. The agent is built to say **"I will not answer that"** and hand over evidence.

Value proven on a real business, not a sandbox

Rizq is a working Saudi restaurant POS with real exported data — 477 orders in the development fixture. The agent found **two genuine defects in the shipping product**, including a silent card-payment failure that produces no log line, plus a false-alarm bug in its own scanner that would have fired every day in production.

Why this is Autonomous Enterprise

The loop that normally requires a company — notice a problem, diagnose it, fix what is safe to fix, prove the fix held, refuse what needs a human, and build the missing capability — runs inside the business's own software, in the owner's own dialect, at zero marginal cost.

What is not done — stated plainly

- **ZATCA is not connected.** VAT and zakat figures are local estimates. Nothing is submitted to the authority. Real e-invoicing needs EGS onboarding, a CSID, compliant signed XML and official sandbox validation.
- **Arabic normalisation.** The three held-out misses all come from regex matching without hamza/prefix normalisation. This is the highest-value next fix — and it must be validated on a *new* held-out set.
- **Browser-local data.** Each device holds its own SQLite. A durable shared backend, automated backups and tested restores are required before multi-device use.
- **PINs stored in clear text** in the POS build (P-4). The new role layer compares them in constant time and never logs them, but it cannot fix the storage without the POS source.
- **The day-book is never written** (P-6). The agent can rebuild it on a copy, but the POS itself does not populate it, and live write-back is not implemented.
- **Compiled source.** UI fixes are reversible bundle patches. They are documented so they can be re-applied as proper source changes when the repository is available.
- **Honest status** — the agent proposes single-file code changes for admin review; it does not autonomously ship production code.

> **Disclosure (organizer requirement).** Runtime models: Qwen3.5 0.8B / Qwen3 8B, run locally through Ollama (Apache 2.0 declared upstream). Built with assistance from Claude (Anthropic). Font: Tajawal under the SIL Open Font License. Browser database: sql.js. Simulator scenarios are synthetic and seed-generated; the Rizq database fixtures are real exports from the live application. Project and third-party distribution licensing are not yet settled, so the complete application is not claimed to be open source. The "Made in KSA" wordmark is custom and is not an official Saudi Made certification.

RIZQ AGENT

A restaurant owner asks in their own words. The software answers, fixes itself, or says no.

- **It works on real data** — held out once and never tuned against: 90% action accuracy on unseen owner requests, 99% on 212 planted database faults, 100% on 300 shifted simulator scenarios.
- **It is safe by construction** — 0 unsafe writes and 0 spurious findings across every evaluation; the source database is never modified.
- **It is honest** — it refuses, escalates, and reports its own limits. The gaps on slide 8 are published, not hidden.
- **It costs nothing to run** — a free local model, Python standard library, no paid API.

Demo & code

Demo video: [link]

Code: [GitHub link]

Run it: *python3 -m rizq_agent web*

Ask it: «كم زكاتي؟»

Mohammed Bakr · Computer Engineering, Taif University

HUMAIN Future AI Builders Hackathon · Challenge 3 · 31 Oct 2026